

Lista 1

Enzo Mialich RA 167097
João Victor Gomes Carneiro RA 239738

I. QUESTÃO 1

A. Caso 1 (NOT A) AND B

A	$\bar{A}$	B	$\bar{A} \wedge B$
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0

Para esse caso, temos que $f(x) = 1|0$:

$$f(x) = 1 \rightarrow \omega^T * x + b \geq 0 \quad (1)$$

$$f(x) = 0 \rightarrow \omega^T * x + b < 0 \quad (2)$$

Sendo que para as diferentes combinações:

$$w_1 * x_1 + w_2 * x_2 + b = ? \quad (3)$$

$$w_1 * x_1 + w_2 * x_2 + b > 0 \iff x_1 = 0 \wedge x_2 = 1$$

$$w_1 * 0 + w_2 * 1 + b = 1 \rightarrow b > -w_2$$

Para outras combinações de entrada $f(x) = 0$:

$$w_1 * 0 + w_2 * 0 + b < 0 \rightarrow b < 0$$

$$w_1 * 1 + w_2 * 1 + b < 0 \rightarrow b < -(w_1 + w_2)$$

$$w_1 * 1 + w_2 * 0 + b < 0 \rightarrow b < -w_1$$

Se $w_1 = -1$ (chute inicial):

$$\begin{cases} b < 1 \\ b < 1 - w_2 \\ b < 0 \\ b > -w_2 \end{cases} \quad (4)$$

$$1 - w_2 < 0 \rightarrow w_2 > 1 \xrightarrow{\text{tomamos}} w_2 = 2$$

$$b \in [-2, -1)$$

Tomando $b = -1.5$, temos:

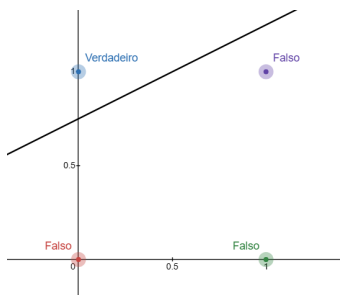


Fig. 1: Solução proposta

$$w = [-1.5 \quad -1 \quad 2] = [w_0 \quad w_1 \quad w_2] \quad (5)$$

B. Caso 2 (A OR B) AND (A OR C)

A	B	C	$A \vee B$	$A \vee C$	$Z = (A \vee B) \wedge (A \vee C)$
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	1	0	0
0	1	1	1	1	1
1	0	0	1	1	1
1	0	1	1	1	1
1	1	0	1	1	1
1	1	1	1	1	1

$$f(x) = 1 \rightarrow \omega^T * x + b \geq 0 \quad (6)$$

$$f(x) = 0 \rightarrow \omega^T * x + b < 0 \quad (7)$$

Os pontos da tabela definem um cubo no R^3 onde os vértices vermelhos indicam as saídas negativas e os verdes a de interesse.

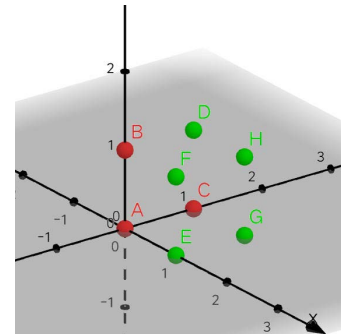


Fig. 2: Pontos no R^3

A análise feita no Geogebra [1] para encontrar um plano que separasse os pontos de interesse na figura 2 leva à equação do plano:

$$1.9x + 0.8y + z = 1.4 \quad (8)$$

Chamando de $f(w_0, w_1, w_2, w_3)$ onde, $w_0 = -1.4, w_1 = 1.9, w_2 = 0.8, w_3 = 1$ onde o vetor de entrada $(x_1, x_2, x_3) = (x, y, z)$ a função definida se resume a:

$$f(w_0, w_1, w_2, w_3) = 1.9x_1 + 0.8x_2 + x_3 - 1.4 \quad (9)$$

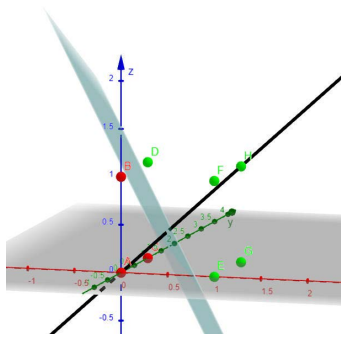


Fig. 3: Plano que separa os pontos no R^3

E aplicando na definição de que:

$$f(x) = 1 \rightarrow \omega^T * x + b \geq 0 \quad (10)$$

$$f(x) = 0 \rightarrow \omega^T * x + b < 0 \quad (11)$$

É facilmente verificado que para as entradas da tabela verdade as saídas são válidas, portanto o sistema é linearmente separável e:

$$w = [-1.4 \quad 1.9 \quad 0.8 \quad 1] = [w_0 \quad w_1 \quad w_2 \quad w_3] \quad (12)$$

C. Caso 3 (A XOR B) AND (A OR C)

A	B	$A \oplus B$	$A \vee B$	$Z = (A \oplus B) \wedge (A \vee B)$
0	0	0	0	0
0	1	1	1	1
1	0	1	1	1
1	1	0	1	0

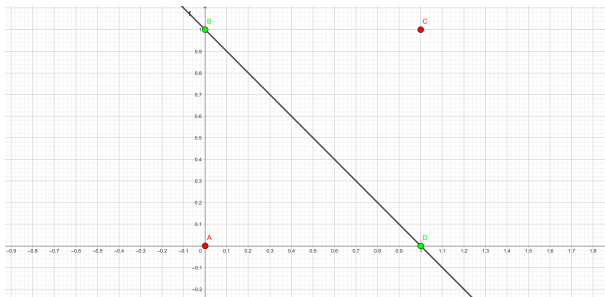


Fig. 4: Pontos no R^2

Visualmente, de acordo com a figura 4, o problema não é linearmente separável pois ambos os pontos negativos estão em extremidades opostas. Matematicamente:

$$0 * 0 + 0 * 0 + w_0 < 0 \rightarrow w_0 < 0 \quad (13)$$

$$0 * 0 + w_2 * 1 + w_0 \geq 0 \quad (14)$$

$$w_1 * 1 + 0 * 0 + w_0 \geq 0 \quad (15)$$

$$w_1 + w_2 + w_0 < 0 \quad (16)$$

Somando as eq. 14 e 15 leva a:

$$w_1 + w_2 + w_3 \geq -2w_0 \quad (17)$$

Mas da eq.16 $w_1 + w_2 + w_0 < 0$ então:

$$w_1 + w_2 + w_0 \geq -2w_0 + w_0 = -w_0 \quad (18)$$

Mas $w_0 < 0$ então a eq.16 se contradiz provando que o sistema não é linearmente separável pois não há w_0, w_1 e w_2 que separe os pontos verdadeiros dos falsos.

II. QUESTÃO 2

A.

Para resolver esse problema, podemos escrever um código simples em python para realizar a multiplicação de cada camada oculta. O loop permite testar cada uma das entradas de forma independente, armazenando os resultados finais na lista "predições". Essa lista, posteriormente, será usada para verificar os erros e acertos da rede.

Não foi necessário aplicar uma função de ativação diretamente, tendo em vista que a função de ativação linear propaga para a saída do neurônio aquela entrada.

`import numpy as np`

`# entradas com bias=1 na 1ª posição`

`entradas = np.array([`

`[1, 0, 0],`

`[1, 1, 1],`

`[1, 1, 0],`

`[1, 0, 1]`

`])`

`# pesos: [bias, w1, w2]`

`pesos = np.array([`

`[-1.0, 1.0, 1.0], # h1`

`[-0.5, 1.0, 1.0], # h2`

`[-0.5, -2.0, 1.0] # y`

`])`

`saidas = np.array([1, 1, 0, 0])`

`predicoes = []`

`linhas = []`

`print(f"{'\hat{y}':>6} {'y':>3} {'cmp':^6}")`

`print("-"*50)`

`for idx, x in enumerate(entradas):`

`h1 = np.dot(x, pesos[0])`

`h2 = np.dot(x, pesos[1])`

`h = np.array([1.0, h1, h2])`

`# adiciona bias para a saída`

`y_hat = float(np.dot(h, pesos[2]))`

`# escalar`

`predicoes.append(y_hat)`

`cmp = "OK" if y_hat == saidas[idx] //`

`else "ERRO"`

```
print(f"{y_hat:>6.2f}  //  
{saidas[idx]:>3}  {cmp:^6}")
```

```
predicoes = np.array(predicoes)
```

```
erros = int(np.sum(predicoes != saidas))  
# conta quantos diferem  
acuracia = (len(saidas) - erros) //  
/ len(saidas)
```

```
print("-"*50)  
print(f"Erros: {erros}  |  //  
Acurácia: {acuracia:.2%}")
```

Saída:

$\hat{y}$	y	cmp
1.00	1	OK
-1.00	1	ERRO
0.00	0	OK
0.00	0	OK

```
Erros: 1  |  Acurácia: 75.00%
```

B.



Fig. 5: Pontos no R^2

Este problema é semelhante ao item C (I-C), onde pode-se ver claramente que o problema não é nem linearmente separável. Além de que não há nenhuma função de ativação nas camadas para que introduza uma não linearidade e dar ao modelo capacidade de curvar o espaço de decisão.

O backpropagation falha em resolver os erros quando uma rede neural é composta exclusivamente de funções de ativação lineares devido a problemas matemáticos fundamentais relacionados ao gradiente e à capacidade de representação da rede.

O problema fundamental do backpropagation com funções lineares está relacionado à derivada constante. Para uma função de ativação linear, a derivada é:

$$f(x) = ax$$

$$\frac{df(x)}{dx} = a$$

Isso torna a regra da cadeia no backpropagation um produto de constantes e matrizes de pesos, sem qualquer modulação dependente do ponto.

III. QUESTÃO 3

A MLP inicial tem a estrutura a seguir:

```
class Net(nn.Module):  
    def __init__(self):  
        super(Net, self).__init__()  
        self.fc1 = nn.Linear(28*28, 50)  
        self.fc1_drop = nn.Dropout(0.2)  
        self.fc2 = nn.Linear(50, 50)  
        self.fc2_drop = nn.Dropout(0.2)  
        self.fc3 = nn.Linear(50, 10)  
  
    def forward(self, x):  
        x = x.view(-1, 28*28)  
        x = F.relu(self.fc1(x))  
        x = self.fc1_drop(x)  
        x = F.relu(self.fc2(x))  
        x = self.fc2_drop(x)  
        return F.log_softmax(self.fc3(x),  
                               dim=1)
```

Ou seja uma rede com 3 camadas com ReLu's entre elas para inserir não-linearidade no modelo. A primeira camada transforma os 784 pixels (28x28) da imagem em 50 neurônios. A camada de Dropout zera aleatoriamente 20% das ativações para evitar overfitting. A segunda camada não reduz a dimensão dos dados e segue a mesma proporção de desativação. Por fim a camada 3 reduz de 50 para 10 neurônios que irão guardar os pesos das 10 classes (dígitos 0-9 do MNIST). Por fim, a rede retorna um vetor de log-probabilidades. Esta rede obteve uma acurácia de 96% no conjunto de teste e demorou 1min 33s para treinamento e validação em 10 épocas com batches de 32 imagens.

A primeira proposta de alteração na rede esta ilustrada a seguir:

```
class Net1(nn.Module):  
    def __init__(self):  
        super(Net1, self).__init__()  
        # Camada de normalização  
        self.norm = nn.BatchNorm2d(1)  
  
        # Aplica Zero Padding --> Colocando 0's nas bordas  
        # da imagem  
        self.pad = nn.ZeroPad2d(1)  
  
        # Aumenta linha e coluna por causa do padding  
        self.fc1 = nn.Linear((28+2)*(28+2), 50)  
        self.fc1_drop = nn.Dropout(0.2)  
        self.fc2 = nn.Linear(50, 50)  
        self.fc2_drop = nn.Dropout(0.2)  
        self.fc3 = nn.Linear(50, 10)  
  
    def forward(self, x):  
        # Garante formato [batch_size, C, H, W]  
        # com C=1 --> devido ao BatchNorm2d  
        if x.dim() == 3: # [batch_size, H, W]  
            # [batch_size, 1, H, W]  
            x = x.unsqueeze(1)  
        # Aplica o padding
```

```

# [batch_size, 1, 30, 30]
x = self.pad(x)
# Aplica a normalização
x = self.norm(x)
# Continua o mesmo processo
# [batch_size, 30*30] --> Mesma coisa que
#x.view(-1, 30*30)
x = torch.flatten(x, 1)
x = F.relu(self.fc1(x))
x = self.fc1_drop(x)
x = F.relu(self.fc2(x))
x = self.fc2_drop(x)
return self.fc3(x)

```

A ideia aqui foi explorar a utilização do Padding na imagem (adicionar 0's nas bordas), já que por se tratar de uma imagem de apenas 1 canal, preto e branco, aqui os zeros poderiam "atrapalhar" a detecção dos dígitos, já que a cor preta está presente nos mesmos. A adição da camada de normalização é com intuito de deixar os valores dos pixels mais estáveis, a fim de acelerar e estabilizar os treinos, essa operação não altera a dimensão do que passa por ela. Note que agora as dimensões da primeira camada não é mais a original, e sim 30x30, devido a adição dos zeros nas bordas. O restante da rede permaneceu inalterado. O desempenho dessas alterações foi sutil com relação a desempenho e tempo de treinamento, aumentando a acurácia apenas para 97.36% e mantendo o mesmo tempo de treinamento.

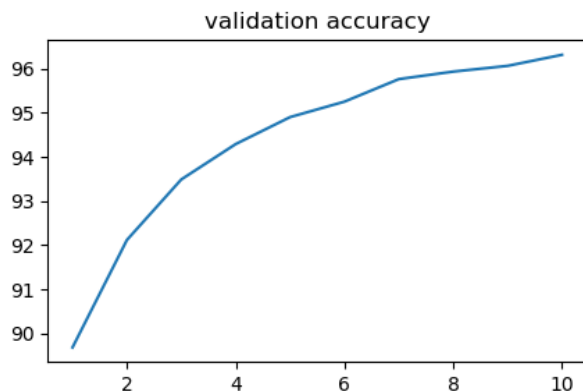


Fig. 6: Validação Net

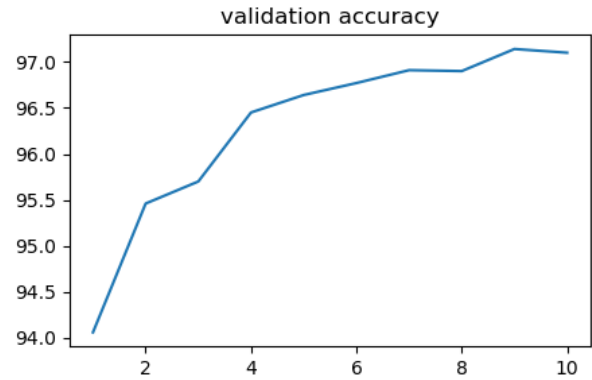


Fig. 7: Validação Net1

O único detalhe que podemos observar foi o contrário do esperado, no quesito estabilidade, na Fig 6 a curva de validação tende para seu valor final de maneira mais suave. Já a do primeiro modelo proposto, Fig 7 tem variações durante as épocas de treinamento.

O modelo 2 seguiu a mesma ideia do original, apenas incluindo mais uma camada e aumentando o número de neurônios:

```

class Net2(nn.Module):
    def __init__(self):
        super(Net2, self).__init__()
        # Criar mais camadas e com mais neurônios
        self.fc1 = nn.Linear(28*28,
                               (28*28)//2)
        self.fc1_drop = nn.Dropout(0.2)
        # Divide novamente por 2 o número de neurônios
        # com base no tamanho da imagem
        self.fc2 = nn.Linear((28*28)//2,
                               (28*28)//4)
        self.fc2_drop = nn.Dropout(0.2)
        # Volta para estrutura anterior
        self.fc3 = nn.Linear((28*28)//4, 50)
        self.fc3_drop = nn.Dropout(0.2)
        self.fc4 = nn.Linear(50, 10)

    def forward(self, x):
        x = x.view(-1, 28*28)
        x = F.relu(self.fc1(x))
        x = self.fc1_drop(x)
        x = F.relu(self.fc2(x))
        x = self.fc2_drop(x)
        x = F.relu(self.fc3(x))
        x = self.fc3_drop(x)
        return F.log_softmax(self.fc4(x), dim=1)

```

Agora a rede passa a ter 4 camadas com 392, 196, 50 e 10 neurônios, respectivamente, em cada. Esse acréscimo de parâmetros aumenta de 42310 para 395108 parâmetros treináveis, aproximadamente 9.4x mais que a *Net*. Esse acréscimo tende a aumentar o tempo e desempenho de treinamento porém pode ocasionar overfitting. Os resultados demonstraram um aumento para 97.84% no desempenho e redução de 0.02s no tempo de treinamento.

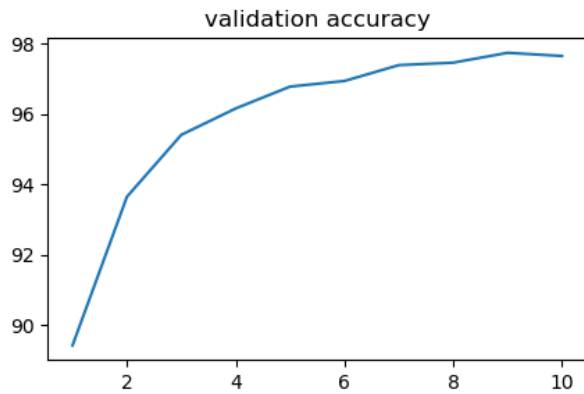


Fig. 8: Validação Net2

A Fig 8 apresenta menos "instabilidades" quando comparada com a da *Net1* e também uma evolução mais rápida no desempenho quando comparada o modelo original da *Net*.

O modelo 3 da rede:

```
class MLP98(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(28*28, 784)
        self.bn1 = nn.BatchNorm1d(784)
        self.drop1 = nn.Dropout(0.2)

        self.fc2 = nn.Linear(784, 256)
        self.bn2 = nn.BatchNorm1d(256)
        self.drop2 = nn.Dropout(0.2)

        self.fc3 = nn.Linear(256, 128)
        self.bn3 = nn.BatchNorm1d(128)
        self.drop3 = nn.Dropout(0.2)
    # CrossEntropyLoss inclui Softmax
        self.fc4 = nn.Linear(128, 10)

    def forward(self, x):
        x = x.view(x.size(0), -1) # [N, 784]
        x = F.relu(self.bn1(self.fc1(x)))
        x = self.drop1(x)
        x = F.relu(self.bn2(self.fc2(x)))
        x = self.drop2(x)
        x = F.relu(self.bn3(self.fc3(x)))
        x = self.drop3(x)
        return self.fc4(x)
```

```
model3 = MLP98().to(device)
optimizer3 = torch.optim.Adam(model3.parameters(),
    lr=1e-3, weight_decay=1e-4) # Penalização L2
```

Cria uma MLP mais larga nas primeiras camadas onde a informação das imagens dos dígitos ainda estão brutas, o *BatchNorm1d* tem a função de manter a estabilidade e facilitar o cálculo do gradiente. Como no caso dessa base de dados, ser extensa, mas não o suficiente para batches de treinamento/validação maiores, foi mantido o mesmo valor do utilizado anteriormente nas outras estruturas propostas. Uma

das principais mudanças, além da estrutura da rede, foi a troca do otimizador, que agora o escolhido é o *Adam* ao invés do *SGD* utilizado anteriormente. Para esta configuração obtivemos um desempenho 98.25% e um tempo total de 2min 33s.

IV. QUESTÃO 4

Sim, quando tiver várias camadas ocultas, sua profundidade será determinada pela quantidade de camadas entre a entrada e saída da rede. A principal vantagem da MLP é a característica de ser totalmente conectada, podendo modelar qualquer relação entre entrada e saída, porém essa mesma vantagem traz junto uma desvantagem, que devido ser totalmente conectada terá muito mais parâmetros quando comparada a uma rede convolucional tradicional. Isso deixa o treinamento mais pesado devido a maior quantidade de pesos para ajuste e deixa a rede sujeita a overfitting dos dados.

V. QUESTÃO 5

05

$$I = \begin{bmatrix} 0,1 & 0,7 & 0,6 & 0,5 \\ 0,4 & 0,8 & 0,7 & 0,8 \\ 0,4 & 0,6 & 0,7 & 0,5 \end{bmatrix}$$

$$K = \begin{bmatrix} 0,1 & 0,3 \\ 0,2 & 0,1 \end{bmatrix}$$

a) $I^* = I * K$

01 $\begin{bmatrix} 0,1 & 0,7 \\ 0,4 & 0,8 \end{bmatrix} * \begin{bmatrix} 0,1 & 0,3 \\ 0,2 & 0,1 \end{bmatrix} = 0,38$

02 $\begin{bmatrix} 0,7 & 0,6 \\ 0,8 & 0,7 \end{bmatrix} * \begin{bmatrix} 0,1 & 0,3 \\ 0,2 & 0,1 \end{bmatrix} = 0,48$

03 $\begin{bmatrix} 0,6 & 0,5 \\ 0,7 & 0,8 \end{bmatrix} * \begin{bmatrix} 0,1 & 0,3 \\ 0,2 & 0,1 \end{bmatrix} = 0,43$

04 $\begin{bmatrix} 0,4 & 0,8 \\ 0,9 & 0,6 \end{bmatrix} * \begin{bmatrix} 0,1 & 0,3 \\ 0,2 & 0,1 \end{bmatrix} = 0,52$

05 $\begin{bmatrix} 0,5 & 0,7 \\ 0,6 & 0,7 \end{bmatrix} * \begin{bmatrix} 0,1 & 0,3 \\ 0,2 & 0,1 \end{bmatrix} = 0,48$

06 $\begin{bmatrix} 0,7 & 0,8 \\ 0,7 & 0,5 \end{bmatrix} * \begin{bmatrix} 0,1 & 0,3 \\ 0,2 & 0,1 \end{bmatrix} = 0,5$

$$I^* = \begin{bmatrix} 0,38 & 0,48 & 0,43 \\ 0,52 & 0,48 & 0,5 \end{bmatrix}$$

b)

$$I = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0,1 & 0,7 & 0,6 & 0,5 & 0 \\ 6 & 0,4 & 0,8 & 0,7 & 0,8 & 0 \\ 6 & 0,9 & 0,6 & 0,7 & 0,5 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad K = \begin{bmatrix} 0,1 & 0,3 \\ 0,2 & 0,1 \end{bmatrix}$$

$$I^* = \begin{bmatrix} 0,01 & 0,09 & 0,2 & 0,17 & 0,1 \\ 0,04 & 0,38 & 0,48 & 0,43 & 0,21 \\ 0,22 & 0,52 & 0,48 & 0,5 & 0,18 \\ 0,09 & 0,27 & 0,27 & 0,22 & 0,05 \end{bmatrix}$$

Ele mantém o produto anterior com o kernel no centro e adiciona nas bordas, valores ponderados pelo kernel, da imagem original. Isso é interessante pois pode por exemplo aumentar informações relevantes da imagem nas bordas e nos cantos resultando em uma melhor generalização para uma rede neural por exemplo.

c)

$$I^* = \begin{bmatrix} 0,01 & 0,09 & 0,2 & 0,17 & 0,1 \\ 0,07 & 0,38 & 0,48 & 0,43 & 0,21 \\ 0,22 & 0,52 & 0,48 & 0,5 & 0,18 \\ 0,09 & 0,27 & 0,27 & 0,22 & 0,05 \end{bmatrix}$$

$$\text{Max Pooling}(I^*) = \begin{bmatrix} 0,38 & 0,48 & 0,48 & 0,43 \\ 0,52 & 0,52 & 0,48 & 0,5 \\ 0,52 & 0,52 & 0,48 & 0,5 \end{bmatrix}$$

Alguns caracteres relevantes da imagem.

d) Max Pooling(I^* , stride = 2)

Neste caso seria retornado um erro pois a dimensão da matriz não é do tamanho suficiente para realizar o Max Pooling com stride 2, falta uma coluna.

VI. QUESTÃO 6

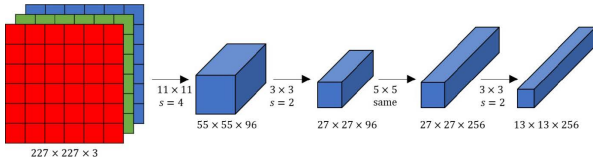


Fig. 9: Camadas AlexNet

A biblioteca do Pytorch [2] implementa a classe **Conv2d** que implementa a convolução em imagens de duas dimensões e diferentes canais. A biblioteca fornece os parâmetros da entrada e da saída:

$$\text{Input: } (N, C_{in}, H_{in}, W_{in}) \quad (19)$$

$$\text{Output: } (N, C_{out}, H_{out}, W_{out}) \quad (20)$$

Na equação(19), os hiperparâmetros N , C_{in} , H_{in} e W_{in} são o *batch size* (quantidade de imagens passadas por vez), número de canais, altura e largura da imagem, respectivamente. Então, ela fornece a relação das dimensões da saída relacionando os hiperparâmetros passados pelo usuário ao utilizar a classe. As dimensões da saída serão dadas por:

$$H_{out} = \frac{H_{in} + 2 \times padding}{stride} - \frac{dilation \times (kernel_size - 1) - 1}{stride} + 1 \quad (21)$$

$$W_{out} = \frac{W_{in} + 2 \times padding}{stride} - \frac{dilation \times (kernel_size - 1) - 1}{stride} + 1 \quad (22)$$

Nas equações (21-22) foi considerado que o *kernel_size* e o *stride*, ou seja, a matriz de pesos do kernel é quadrada e o passo será o mesmo nas duas direções da imagem. Considerando que *dilation* = 1 (valor usual) e *padding* = 0, abreviando as equações, podemos chegar na equação em função de k e s :

$$H_{out}|W_{out} = \frac{H_{in}|W_{in} - k}{s} + 1 \quad (23)$$

Os valores 96 e 256, na imagem 9 indicam a quantidade de filtros que a rede possui para terem seus pesos treinados, ou seja, na primeira e segunda camada possuem 96 e 256 filtros, respectivamente. O código em Python a seguir faz essa busca:

```
# Tamanho da entrada e saída entre camadas
in_out = [(227, 55), (55, 27), (27, 13)]
```

```
resultados = []
#Itera sobre o array
for v in in_out:
    # Calcula o stride como sendo um valor
    #inteiro da relação in/out
    s = v[0] // v[1]
```

```
#procura um k num range aleatório de 1-15
for k in range(1,15):
    # Isola a equação do calculo da saída
    eq = (v[1]-1)*s - (v[0]-k)
    if eq ==0:
        resultados.append((k,s))
        break
print(resultados)
```

Saída:

[(11, 4), (3, 2), (3, 2)]

Os resultados mostram que na entrada a imagem passa pela primeira camada onde possui um *kernel_size* = (11 × 11) com $s = 4$, da primeira para a segunda um *kernel_size* = (3 × 3) com $s = 2$, da segunda para a terceira o tamanho da imagem na saída da camada permaneceu o mesmo da anterior, a classe *Conv2d* além de um valor inteiro para o *padding*, possui outra opção chamada "same", o que levaria a outro loop interno para testar esse valor, mas em [3] os autores explicam e demonstram que *padding* = 2 nas equações (21-22), que substituindo no código levaria a *kernel_size* = (5 × 5) com $s = 1$ e $p = 2$. Por fim para ultima camada o código retorna *kernel_size* = (3 × 3) com $s = 2$.

VII. QUESTÃO 7

A **Visual Geometry Group (VGG)** é uma típica Convolutional Neural Network (CNN) profunda com diversas camadas, as nomenclaturas, VGG-16 ou VGG-19 por exemplo, indicam quantas camadas convolucionais a rede possui [4]. As suas principais características quando comparada com outras redes convolucionais são:

- Utiliza filtros pequenos (3×3) em todas as camadas convolucionais.
- Estrutura simples e uniforme: várias camadas convolucionais empilhadas, seguidas por camadas totalmente conectadas no final.
- Prós: fácil de entender e servir de base para outros modelos.
- Contras: número muito grande de parâmetros → exige bastante memória e poder de processamento.

A imagem 10 ilustra a estrutura da VGG, mostrando a característica das camadas conectadas e empilhadas.

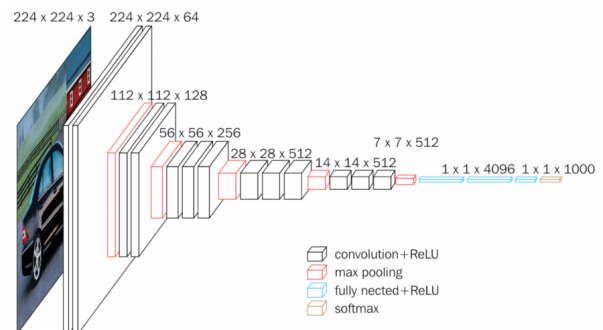


Fig. 10: Arquitetura VGG

A **Rede Neural Residual (ResNet)** é uma arquitetura de aprendizado profundo na qual as camadas aprendem funções residuais com referência às entradas da camada. Foi desenvolvida em 2015 para reconhecimento de imagem e venceu o ImageNet Large Scale Visual Recognition Challenge (ILSVRC) daquele ano [5]. A formulação matemática pode ser descrita sucintamente por:

$$F(x) = H(x) \quad (24)$$

$$y = F(x) + x \quad (25)$$

Em um modelo de rede neural multicamadas, considere uma sub-rede com um certo número de camadas empilhadas (por exemplo, 2 ou 3). Denote a função subjacente desempenhada por esta sub-rede como $H(x)$, onde x é a entrada para a sub-rede. O aprendizado residual reparametriza esta sub-rede e permite que as camadas de parâmetros representem uma "função residual". A operação de $+x$ é implementado por meio de uma "conexão de salto" que realiza um mapeamento de identidade para conectar a entrada da sub-rede com sua saída. Essa conexão é chamada de "conexão residual" em trabalhos posteriores. A função $F(x)$ é frequentemente representada por multiplicação de matrizes entrelaçadas com funções de ativação e operações de normalização (por exemplo, normalização em lote ou normalização em camada).

Suas principais características podem ser descritas como:

- Introduziu os atalhos (skip connections) ou blocos residuais, que "pulam" camadas.
- Resolve o problema do desvanecimento/explosão do gradiente, permitindo treinar redes muito profundas (com centenas de camadas).
- Prós: maior profundidade sem perda de desempenho, mais eficiente que redes tradicionais muito grandes.
- Contras: mais complexa de implementar e entender em comparação à VGG.

VIII. QUESTÃO 8

Alterações discutidas no notebook da questão em anexo.

REFERENCES

- [1] "_PTGeoGebra Clássico." [Online]. Available: https://www.geogebra.org/classic?lang=pt_PT
- [2] "Conv2d — PyTorch 2.8 documentation." [Online]. Available: <https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv2d.html>, <https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv2d.html>
- [3] J. Silva, "AlexNet para classificação de imagens," Dec. 2020. [Online]. Available: <https://jefferson023.medium.com/alexnet-para-classifica%C3%A7%C3%A3o-de-imagens-d86c482a44b3>
- [4] "Residual neural network," Aug. 2025, page Version ID: 1304566021. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Residual_neural_network&oldid=1304566021
- [5] "Residual neural network," Aug. 2025, page Version ID: 1304566021. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Residual_neural_network&oldid=1304566021